

Configure IPv6 Addressing

Answers to Tasks 1 to 5 (Windows and Linux commands). Replace **Ethernet** / **eth0** with your own adapter name.

Task 1: Assign a static IPv6 address and verify

Use the command line to assign a static IPv6 address to the primary adapter and verify it.

Windows (run Command Prompt as Administrator)

```
netsh interface ipv6 show interfaces
netsh interface ipv6 add address "Ethernet" 2001:db8:1::10/64
ipconfig
netsh interface ipv6 show address "Ethernet"
```

Linux

```
ip link show
sudo ip -6 addr add 2001:db8:1::10/64 dev eth0
ip -6 addr show dev eth0
ifconfig eth0
```

Verification: the output of ipconfig (Windows) or ip -6 addr (Linux) should list 2001:db8:1::10 with prefix length 64 under the adapter. 2001:db8::/32 is the documentation prefix, so it is safe for practice.

Task 2: Unique Local Address (fd00::/8) and ping test

Configure a ULA starting with 'fd' and ping another LAN device that also has a ULA.

Both devices must be in the same /64. Example: Device A = fd12:3456:789a:1::1, Device B = fd12:3456:789a:1::2.

Windows

```
netsh interface ipv6 add address "Ethernet" fd12:3456:789a:1::1/64
ping -6 fd12:3456:789a:1::2
```

Linux

```
sudo ip -6 addr add fd12:3456:789a:1::1/64 dev eth0
ping6 fd12:3456:789a:1::2
# or: ping -6 fd12:3456:789a:1::2
```

Expected result: replies such as Reply from fd12:3456:789a:1::2: time<1ms. If the ping fails, allow ICMPv6 in the firewall (Windows: enable 'File and Printer Sharing (Echo Request - ICMPv6-In)') and confirm both devices are on the same LAN/VLAN.

Task 3: Temporary IPv6 address and behavior after reboot / reconnect

Set up a temporary address and check if it changes. Document steps and observations in a text file.

Steps

```
# Windows - enable privacy (temporary) addresses
netsh interface ipv6 set privacy state=enabled
netsh interface ipv6 show privacy
Get-NetIPAddress -AddressFamily IPv6 | Select IPAddress,PrefixOrigin,SuffixOrigin

# Linux - enable temporary addresses
```

```
sudo sysctl -w net.ipv6.conf.eth0.use_tempaddr=2
ip -6 addr show dev eth0      # look for the word "temporary"

# Record the address, then test:
# 1) Disable and re-enable the adapter (or Wi-Fi off/on)
# 2) Reboot the system
# 3) Run the show command again and compare
```

Observation file (temp_ipv6_notes.txt) - sample format

```
Date/Time      : <fill in>
Adapter        : Ethernet / eth0
Before test    : Temporary addr = 2001:db8:1:0:a1b2:c3d4:e5f6:1122
After reconnect: Temporary addr = <fill in>
After reboot   : Temporary addr = <fill in>
Conclusion     : Temporary (privacy) address is randomly generated and
                  changes after reboot and when it expires. The stable
                  address (EUI-64 / stable-privacy) remains the same.
```

Expected behavior: a temporary (privacy) address is randomly generated, has a short preferred lifetime, and usually changes after a reboot or after the old one expires. A reconnect may or may not change it. Also note that an address added manually with `ip addr add` or `netsh ... store=active` is lost on reboot. Write your *actual* observed values in the file.

Task 4: Zomato-style restaurant search (Python)

Store restaurant names with IPv6 addresses; partial-name input returns the matching IPv6 address.

```
# restaurant_search.py
restaurants = {
    "Pizza Palace": "2001:db8:1::10",
    "Spice Garden": "2001:db8:1::11",
    "Burger Barn": "2001:db8:1::12",
    "Pasta Point": "2001:db8:1::13",
    "Sushi Central": "2001:db8:1::14",
}

def search(partial):
    partial = partial.strip().lower()
    return [(n, ip) for n, ip in restaurants.items() if partial in n.lower()]

while True:
    text = input("Enter restaurant name (or 'exit'): ")
    if text.lower() == "exit":
        break
    if not text.strip():
        print("Please type something.")
        continue
    results = search(text)
    if results:
        for name, ip in results:
            print(f"{name} -> {ip}")
    else:
        print("No restaurant found.")
```

Sample run:

```
Enter restaurant name (or 'exit'): pa
Pizza Palace -> 2001:db8:1::10
Pasta Point -> 2001:db8:1::13
Enter restaurant name (or 'exit'): zzz
No restaurant found.
Enter restaurant name (or 'exit'): exit
```

Run: python restaurant_search.py. The search is case-insensitive and returns every restaurant whose name contains the typed text.

Task 5: AI-generated subnetting scenario and manual assignment

Paste the AI-generated scenario and your configuration steps.

Prompt given to ChatGPT / Copilot

"Create an IPv6 subnetting scenario for a small company with at least 3 subnets, using the 2001:db8:abcd::/48 prefix."

Sample AI-generated scenario

A company owns 2001:db8:abcd::/48 and divides it into /64 subnets, one per department:

Subnet	Department	Network	Usable host range
1	Sales	2001:db8:abcd:1::/64	2001:db8:abcd:1::1 - ...:1:ffff:ffff:ffff:ffff
2	Engineering	2001:db8:abcd:2::/64	2001:db8:abcd:2::1 - ...:2:ffff:ffff:ffff:ffff
3	Guest Wi-Fi	2001:db8:abcd:3::/64	2001:db8:abcd:3::1 - ...:3:ffff:ffff:ffff:ffff

Note: /48 to /64 gives 16 bits of subnet space (65,536 possible subnets); every /64 holds 2^{64} addresses.

(Tip: for your submission, run the prompt yourself and paste the real ChatGPT/Copilot output here, since it may differ from this sample.)

My configuration (Engineering subnet)

Chosen address : 2001:db8:abcd:2::50/64

Windows:

```
netsh interface ipv6 add address "Ethernet" 2001:db8:abcd:2::50/64  
ipconfig
```

Linux:

```
sudo ip -6 addr add 2001:db8:abcd:2::50/64 dev eth0  
ip -6 addr show dev eth0
```

Verification: the adapter shows 2001:db8:abcd:2::50 with prefix /64. The address lies inside Subnet 2 (Engineering) because its first four groups match 2001:db8:abcd:2.